# Literature survey: prompt injection, agent defences, tamper-evident logging, LLMs in telecom

This survey covers the research our trust layer builds on, and where our work differs from it. The report has a short version (Section 1 "Related work" and Section 2 "Security techniques"). This file is the full one.

**How the references were checked (9 Nov 2026).** Each paper below was checked against an official record before we cited it:

- the arXiv API, for title, authors and abstract;
- Crossref, for the DOI, venue and pages;
- the USENIX website, for the three USENIX papers, which have no DOIs.

What we say about each paper comes from its abstract or its full text. We did not paraphrase from memory. No reference was invented.

---

## 1. The threat: prompt injection, direct and indirect

| # | Paper | What it showed | What it means for us |
|---|---|---|---|
| R1 | Perez & Ribeiro, *Ignore Previous Prompt: Attack Techniques for Language Models*, NeurIPS ML Safety Workshop 2022, arXiv:2211.09527 | Simple hand-written inputs can make GPT-3 drop its task (goal hijacking) or reveal its prompt (prompt leaking). | Direct prompt injection: the attacker types to the model. |
| R2 | Greshake, Abdelnabi, Mishra, Endres, Holz, Fritz, *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*, ACM AISec '23, pp. 79–90, doi:10.1145/3605764.3623985 | Defined **indirect prompt injection**: attackers plant prompts in data the application will later retrieve. They showed it working against Bing's GPT-4 chat and code-completion engines. "LLM-integrated applications blur the line between data and instructions." | This is exactly our threat. A NOC agent retrieves alarms, syslog and tickets that outsiders can write. |
| R3 | Vassilev et al., *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, NIST AI 100-2 E2025, doi:10.6028/NIST.AI.100-2e2025 | NIST's official vocabulary for attacks on AI systems, including direct and indirect prompt injection. | We use its terms so that regulators and engineers read our report the same way. |
| R4 | OWASP, *Top 10 for LLM Applications 2025*, LLM01 Prompt Injection | Industry ranking: prompt injection is risk no. 1. | Used in the gap register as the industry baseline. |

## 2. Measuring the threat: benchmarks for LLM agents

| # | Paper | What it showed | What it means for us |
|---|---|---|---|
| R5 | Liu, Jia, Geng, Jia, Gong, *Formalizing and Benchmarking Prompt Injection Attacks and Defenses*, USENIX Security 2024, pp. 1831–1847 | A formal framework for prompt-injection attacks, in which existing attacks are special cases. It designs a combined attack and evaluates 5 attacks and 10 defences on 10 LLMs and 7 tasks. | Our 24 payloads mix the attack styles it formalises: instruct, fake authority, fake evidence, data requests. |
| R6 | Yi et al., *Benchmarking and Defending against Indirect Prompt Injection Attacks on LLMs* (BIPIA), KDD 2025, pp. 1809–1820, doi:10.1145/3690624.3709179 | The first benchmark for indirect injection; the LLMs it tested were "universally vulnerable". Two causes: models cannot tell **information from instructions**, and they do not know to avoid executing instructions found in external content. | The first cause is why we withhold free text completely, instead of asking the model to ignore it. |
| R7 | Zhan, Liang, Ying, Kang, *InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents*, Findings of ACL 2024, pp. 10471–10506, doi:10.18653/v1/2024.findings-acl.624 | 1,054 cases and 30 agents tested. A ReAct-prompted GPT-4 agent was attacked successfully 24% of the time, and adding a "hacking prompt" nearly doubled the rate. | Our raw HolmesGPT result (19% on gpt-4.1, 42% on gpt-4.1-mini) is in the same range, on a different task and domain. |
| R8 | Debenedetti, Zhang, Balunović, Beurer-Kellner, Fischer, Tramèr, *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*, NeurIPS 2024 (Datasets & Benchmarks), pp. 82895–82920, doi:10.52202/079017-2636 | 97 tasks and 629 security cases (email, banking, travel) in an extensible environment for attacks, defences and **adaptive attacks**. | No public benchmark covers network operations. We offer our 24 payloads, adaptive attacks and 36 blind cases as a seed (gap register, SG17). |
| R9 | Zhan, Fang, Panchal, Kang, *Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents*, Findings of NAACL 2025, pp. 7101–7117, doi:10.18653/v1/2025.findings-naacl.395 | Eight defences tested; adaptive attacks bypassed **all eight**, each with over 50% success. | This is why we attacked our own layer adaptively (v1 broke 6/6, v2 and v3 held 0/12) and then ran a blind holdout. |

## 3. Defences

### 3a. Defences inside the model or the prompt (probabilistic)

| # | Paper | Idea | Limit, in our setting |
|---|---|---|---|
| R10 | Hines et al., *Defending Against Indirect Prompt Injection Attacks With* | Mark untrusted input (delimiting, datamarking, encoding) so the model can tell where text came from. On | It lowers the rate but does not remove it, and the model still reads the attacker's sentence. HolmesGPT's own guardrail is |

| # | Paper | Idea | Limit, in our setting |
|---|---|---|---|
| | *Spotlighting*, arXiv:2403.14720, 2024 | GPT-family models, attack success fell from over 50% to under 2%. | a prompt-level defence of this kind, and in our tests it did not help (21% vs 19% raw). |
| R11 | Chen, Piet, Sitawarin, Wagner, *StruQ: Defending Against Prompt Injection with Structured Queries*, USENIX Security 2025, pp. 2383–2400 | Separate the prompt and the data into two channels, and fine-tune the model to follow only the prompt channel. | It needs a specially trained model. An operator using a vendor's model, or HolmesGPT, cannot retrain it. |
| R12 | Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, arXiv:2404.13208, 2024 | Train the model to give system instructions priority over lower-privileged text. | It is built into the model. It improves robustness but gives no guarantee, and a regulator cannot inspect it. |

**3b. Defences in the system around the model (by design)**

| # | Paper | Idea | How our layer relates |
|---|---|---|---|
| R13 | Beurer-Kellner et al., *Design Patterns for Securing LLM Agents against Prompt Injections*, arXiv:2506.08837, 2025 | Six design patterns that give **provable** resistance by limiting what untrusted text can reach: Action-Selector, Plan-Then-Execute, LLM Map-Reduce, **Dual LLM**, Code-Then-Execute, **Context-Minimization**. | Our quarantine is an instance of the **Dual LLM** pattern: the reader of untrusted text has no tools (and in our default "structured" mode it is not a model at all). It also uses **Context-Minimization**: only allow-listed fields and neutral claim templates pass. |
| R14 | Debenedetti et al., *Defeating Prompt Injections by Design* (CaMeL), arXiv:2503.18813, 2025 | It extracts control flow and data flow from the trusted query, so untrusted data "can never impact the program flow". Capabilities and policies are checked at each tool call. It solves 77% of AgentDojo tasks with provable security, vs 84% undefended. | It has the same principle as our rule-checker: a policy check in code at the point of action. CaMeL tracks data flow in general agent programs. We instead **verify each claim against the operator's records** and make the policy **cite the law**. |

**Where our work sits.** Studies such as BIPIA (R6), InjecAgent (R7) and adaptive attacks (R9) agree on two points. Model-level defences lower attack rates but do not remove them, and adaptive attackers get around them. System-level designs (R13, R14) give guarantees by construction. We follow the system-level line and add three things that we did not find in these papers: 1. **Claim verification against an independent system of record.** A claim such as "CHG-4463 approved" is checked against the change log, and comes back verified, contradicted or unverified. The checks run whether or not the detector flagged anything. 2. **A policy node that cites the law and labels its force** (law, licence condition, regulation, guidance, principle, operator policy). Policy use is therefore visible on every decision, as ITU's D10 dimension asks. 3. **A domain-specific evaluation.** We test on network-operations telemetry with an unmodified CNCF agent (HolmesGPT), compare against that agent's own guardrail and summariser, and run a blind holdout written without access to our rules.

**What we did not do (honest).** We did not run AgentDojo or CaMeL ourselves, so we make no head-to-head claim against them. Our numbers come from our own synthetic incident and 36 blind cases.

## 4. Classic security principles we apply

| # | Source | Principle | Where it appears in our system |
|---|---|---|---|
| R15 | Saltzer & Schroeder, *The Protection of Information in Computer Systems*, Proc. IEEE 63(9):1278–1308, 1975, doi:10.1109/PROC.1975.9939 | **Least privilege** | Agents can only propose or advise; none can execute. The quarantine reader has no tools. |
| | | **Complete mediation** | Every proposed action passes the rule-checker; no path bypasses it. |
| | | **Fail-safe defaults** | Unverified means not approved; an unclear case is escalated, not allowed. When rules disagree, the strictest wins (BLOCK > ESCALATE > ALLOW). |
| | | **Separation of privilege** | The model proposes, code decides and a named human approves escalations. |
| | | **Economy of mechanism** | The decision logic is ~4 rules in plain code (`trustlayer/rulechecker.py`, `rules.yaml`), small enough to review by hand. |

## 5. Tamper-evident audit logs

| # | Paper | Idea | Our use, and its limits |
|---|---|---|---|
| R16 | Haber & Stornetta, *How to Time-Stamp a Digital Document*, J. Cryptology 3(2):99–111, 1991, doi:10.1007/BF00196791 | Link each record to the hash of the one before it. Changing any old record breaks every link after it. | Each audit entry stores `prev_hash`, and its own `hash` is SHA-256 over canonical JSON (sorted keys). |
| R17 | Schneier & Kelsey, *Secure Audit Logs to Support Computer Forensics*, ACM TISSEC 2(2):159–176, 1999, doi:10.1145/317087.317089 | Log entries an attacker who later takes over the machine cannot read or undetectably change. It uses evolving keys, MACs and encryption. | We encrypt the stored source text (Fernet) and keep only its SHA-256 fingerprint in the chain. **Not implemented:** their evolving-key MACs (forward integrity). |
| R18 | Crosby & Wallach, *Efficient Data Structures for Tamper-Evident Logging*, USENIX | Tamper-evident logs with an untrusted logger, where outside auditors check | Our `anchor()` copies the latest hash to a separate file the agents never write to, and `tools/verify_audit.py` imports none of our code. |

| # | Paper | Idea | Our use, and its limits |
|---|---|---|---|
| | Security 2009 | commitments. | Without anchors held by another party, someone with full write access could rebuild the whole chain. A production system should publish anchors externally or sign them (planned: Ed25519). |
| R19 | NIST FIPS 180-4, *Secure Hash Standard*, 2015 | The SHA-256 specification. | Hash function for the chain and the source fingerprints. |

**What the audit log is, and what it is not.** It is a **tamper-evident record** of every step: input, claims, verdicts, rules, citations and agent opinions. The original untrusted text is **encrypted at rest** (Fernet: AES-128-CBC with HMAC-SHA256) and referenced by its SHA-256 fingerprint, so the log can be shared without spreading the attacker's text. It does **not** encrypt or sign messages between agents. In this prototype all agents run in one process on one machine. Signed inter-agent messages (Ed25519) and transport encryption are planned.

### 6. LLMs and agents in networks and telecom

| # | Paper | What it covers | The gap |
|---|---|---|---|
| R20 | Wu et al., *NetLLM: Adapting Large Language Models for Networking*, ACM SIGCOMM 2024, pp. 661–678, doi:10.1145/3651890.3672268 | Adapting one LLM to several networking tasks. | It is about capability. It does not consider untrusted text inside network data. |
| R21 | Zhou et al., *Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities*, IEEE Communications Surveys & Tutorials 27(3):1955–2005, 2025, doi:10.1109/COMST.2024.3465447 | A broad survey of LLMs in telecom: generation, classification, optimisation and prediction. | It shows the field moving toward LLMs in operations. Injection through operational telemetry is not its focus. |
| R22 | HolmesGPT (CNCF Sandbox), github.com/HolmesGPT/holmesgpt, commit 65332c5 | An open-source agent that investigates alerts. | We use it unmodified as the agent under test (Exp. 1). |

**The gap we address.** Telecom AI research has studied what LLMs can do in networks (R20, R21). Prompt-injection research has studied general agents: email, banking, travel (R6–R9). We did not find work that measures indirect prompt injection through **network-operations telemetry** and then enforces **national telecom and data-protection law** on the agent's actions. That combination is our contribution.

---

### Mapping: security techniques and the literature behind each

| Threat (STRIDE-style) | Technique | Our implementation (file) | Literature |
|---|---|---|---|
| Tampering with the | Input isolation; Dual-LLM / | Free text withheld; neutral claim | R2, R6, R13 |

| Threat (STRIDE-style) | Technique | Our implementation (file) | Literature |
|---|---|---|---|
| agent's reasoning (indirect injection) | context minimisation | templates; reader has no tools (trustlayer/quarantine.py, claims.py) | |
| Injection hidden in structured fields | Allow-list (positive) input validation | SAFE_KEY, SAFE_VALUE regex, values ≤ 32 characters, otherwise [withheld] (quarantine.py) | R13 |
| Spoofed facts (fake approval, fake incident) | Data-provenance check against a system of record | 6 claim types → verified / contradicted / unverified (verify_claims.py, data/records/) | new in this work |
| Planted identifiers repeated by the agent | Redaction / tokenisation | Unrecorded IDs → [unrecorded ID #n] | new in this work |
| Spoofed authority ("TDRA directive") | Channel authentication + authorisation register | Registered channel and mandate checked against the mandate register (rule 3) | R15 (separation of privilege) |
| Harmful action (elevation of privilege) | Policy enforcement point; least privilege; complete mediation; fail-safe default | Deterministic rule-checker, BLOCK > ESCALATE > ALLOW (rulechecker.py, rules.yaml) | R14, R15 |
| Repudiation or log tampering | Hash chain, external anchors, independent verifier | SHA-256 over canonical JSON + prev_hash; anchors.txt; tools/verify_audit.py | R16–R19 |
| Disclosure of stored attacker text | Authenticated encryption at rest, content-addressed storage | Fernet (AES-128-CBC + HMAC-SHA256), file named by SHA-256 | R17 |
| Early warning (not a decision) | Signature-based detection | 9 regex rules (detector.py). Flags only: verification runs anyway | R9 (detectors alone are bypassed) |

## Full reference list (as cited in the report)


1. ITU-T Y.3172 (06/2019).
2. ITU, AI Ready: Analysis Towards a Standardized Readiness Framework, 2025.
3. TDRA, UAE IA Regulation v1.1.
4. UAE Federal Decree-Law 45/2021 (PDPL).
5. UAE Federal Law by Decree 3/2003, as amended.
6. TDRA Licence No. 2 of 2026.
7. UAE AI Charter, 2024.
8. Directive (EU) 2022/2555 (NIS2).
9. Regulation (EU) 2024/1689 (AI Act).
10. Regulation (EU) 2016/679 (GDPR).
11. UK Communications Act 2003, s.105K.

12. OWASP Top 10 for LLM Applications 2025.
13. HolmesGPT (R22).
14. This repository.
15. R2.
16. R1.
17. R5.
18. R6.
19. R7.
20. R8.
21. R9.
22. R10.
23. R11.
24. R12.
25. R13.
26. R14.
27. R15.
28. R16.
29. R17.
30. R18.
31. R19.
32. R3.
33. R20.
34. R21.